

Soum Deco v4

Every Workflow Traced End-to-End

A workflow-by-workflow trace of soumdeco-magic-v4-final — every data path is followed from origin to completion to verify it actually works. For each workflow I report: **does it complete, where does it break, what v3 audit finding did v4 fix, and what new regression did v4 introduce**. The Cloudinary leak is closed (good) but the KV-write quota is now 50× over budget (bad) and the admin-secret still ships in every visitor's bundle (unchanged).

HEADLINE FOR V4

Good news: The Cloudinary image-corruption bug is genuinely fixed. The team added `reverseOptimizeUrl()` in two places (`use-catalog.ts:47` + `admin-panel.tsx:1217`) that convert local paths back to Cloudinary URLs before saving to the sheet. They also wrote `scripts/repair-images.py` and ran it — the current `products.json` has 0 local paths (was 44 of 60 in v3). `build-image-manifest.py` now uses the `SCRIPT_DIR` pattern + fails loud on empty manifest. **Bad news:** Two new regressions in `worker/data-sync.js` will burn the KV write quota within 30 minutes of typical traffic: the `/refresh` rate limiter was deleted entirely, AND the hit-counter sampling (1-in-50) was removed — every visitor now triggers a KV write. The admin password is still in the client bundle. `.env.production` was deleted but no `.gitignore` exists, so it can come back. Security findings S-01 through S-23 are mostly unfixed.

16	5	75+	4	50×
Workflows traced	v3 findings fixed	v3 findings still open	New regressions in v4	KV write quota burn

Project	soumdeco-magic-v4-final.zip
Compared to	soumdeco-magic-v3-final.zip (audited 2026-08-19)

Method	Formidable-prompt: trace every workflow end-to-end, no skipping
Audit date	2026-08-19
Position	Honest assessment: a real fix shipped, but 2 new critical bugs introduced

1. Executive Summary — What v4 Fixed and What It Broke

v4 is a focused fix-release. The team addressed exactly one of the five Phase-2 items from my v3 audit (the Cloudinary bug), and did it correctly. They also bumped the polling interval from 60s to 300s (Phase 4 item 4.8). That's a real improvement. But while doing so, they introduced two regressions in worker/data-sync.js that will burn the KV write quota within 30 minutes of real traffic — and they did NOT touch any of the security findings from Phase 1 or Phase 3. The admin password is still in the client bundle. The Worker admin secret is still inlined via NEXT_PUBLIC_. All mutating API routes still have zero authentication. The order route still sends PII in URL params and logs it to server logs. Net result: **the Cloudinary bug is fixed, but the security posture is unchanged and the Worker is now capable of DOSing itself.**

1.1 Scorecard: v3 audit items addressed in v4

Phase	Item	v3 recommendation	v4 status	Verdict
P1.2	Remove .env.production	Delete file, add .gitignore with .env*	File deleted, but NO .gitignore added	PARTIAL
P1.3	Remove adminPassword from bundle	Move to server-side check	Still in brand-config.ts:17 = 'dimou2411@dz'	NOT DONE
P1.4	Auth on mutating APIs	Add middleware / header checks	Zero auth added anywhere	NOT DONE
P1.5	Stop direct admin → Apps Script writes	Route admin writes through Worker	client-sheet.ts unchanged — admin posts directly to Apps Script	NOT DONE
P1.6	Convert product_reset/delete to POST	Stop GET-based mutations	Still GET in sheet.ts:122,136	NOT DONE
P2.1	Reverse-rewrite local paths before save	Convert /images/products/ back to Cloudinary URL before sheet POST	DONE: reverseOptimizeUrl in use-catalog.ts:47 + admin-panel.tsx:1217	DONE ☑
P2.2	Repair existing 44 corrupted rows	One-time Apps Script function to rewrite local paths	DONE: scripts/repair-images.py — products.json now has 0 local paths (was 44/60)	DONE ☑
P2.3	Fix build-image-manifest.py paths	Use SCRIPT_DIR/REPO_ROOT pattern + fail-loud on empty	DONE: both fixes applied (scripts/build-image-manifest.py:14-18,47-51)	DONE ☑
P2.5	Remove buildCloudinaryFallback onError	Cloudinary URL always valid now, fallback not needed	product-image.tsx unchanged — onError fallback still present	NOT DONE
P2.6	Reconcile cron schedule	Pick 1-min or 5-min, update code comments	worker/wrangler.toml still says 1-min cron, code comments still say 5-min	NOT DONE
P3.1-3.9	All authorization gaps	9 items: middleware, rate limiting, PII POST, no-PII logs, encryption, etc.	ZERO of 9 items done	NOT DONE
P4.8	Increase POLL_MS 60s → 300s	Reduce Worker request quota burn	DONE: use-catalog.ts:39 = 300_000 (was 60_000)	DONE ☑
P4.9	Delete LoadingFallback.tsx	Eliminates 3M DOM scans/day	File still exists, still does 3-sec DOM scans	NOT DONE
P4.11	Merge use-stock into use-catalog	Single interval, single fetch	worker-client.ts shrank but fetchStockCsv still exists; use-stock.ts unchanged	PARTIAL
NEW	R-1: /refresh rate limiter removed	(not in v3 audit — v3 had the limiter)	worker/data-sync.js:126-160 deleted. Now unlimited /refresh calls = unlimited KV writes	REGRESSION
NEW	R-2: KV hit-counter sampling removed	(not in v3 audit — v3 sampled 1-in-50)	worker/data-sync.js:515-527 rewritten. Every visitor now writes to KV = 50 × quota burn	REGRESSION
NEW	R-3: clientUpsertProduct no longer treats 302/405 as success	(v3 explicitly handled Apps Script's 302→405 redirect quirk)	client-sheet.ts:199 now only returns res.ok. May break admin saves.	REGRESSION (likely)

Phase	Item	v3 recommendation	v4 status	Verdict
NEW	R-4: products.json.backup-* leaked into public/data	(not in v3)	public/data/products.json.backup-1787141289 (67KB) is downloadable by anyone	REGRESSION

Bottom line for v4

5 of 17 audit items addressed (29%). 4 new regressions introduced. The Cloudinary bug is genuinely fixed — that's the headline win. But the Worker now has a self-DOS capability (R-1 + R-2 combined) that will burn through the free-tier KV write quota within 30 minutes of typical traffic.

Recommendation: do not deploy v4 to production without restoring the /refresh rate limiter and the hit-counter sampling. Those are 30-line fixes that prevent a hard outage.

2. Workflow 1 — Visitor Loads Home Page

2.1 Trace

1. Visitor opens `https://sourmdeco.pages.dev/`
2. Cloudflare Pages serves `index.html` (no SSR — `page.tsx` is 'use client')
3. Browser parses HTML, fetches JS bundle (~180KB gzipped, est.)
4. React mounts → `useCatalog()` fires:
 - 4a. `loadCatalog()` synchronous (`JSON.parse` of `localStorage` — was 200-600ms at 9,500-product scale, but catalog is 61 products now so ~5ms)
 - 4b. If empty: check first-visit → `SEED_PRODUCTS` seeded to `localStorage`
 - 4c. `optimizeCloudinaryUrls` applied to cached products (manifest lookup)
 - 4d. `setProducts(sorted)` + `setHydrated(true)` + `setLoading(false)`
 - 4e. `setTimeout(refresh, 300)` — background fetch
 - 4f. `loadImageManifest()` — fetches `/image-manifest.json` (87 files now)
 - 4g. `retryFailedOrders()` — silent retry of failed orders
5. `refresh()` fires:
 - 5a. (NEW IN v4) Optimistic `setProducts(cached)` — instant first paint
 - 5b. `fetchProducts()` → Worker `?action=catalog` (5s timeout)
 - 5c. Worker reads KV (1 read, ~30ms)
 - 5d. (REGRESSION R-2) `bumpHitCounter()` writes KV on EVERY visitor
 - 5e. Returns stringified products + stock CSV
 - 5f. `optimizeCloudinaryUrls` applied → `setProducts` → `saveCatalog`
6. `scheduleNext()` — `setInterval(refresh, 300_000)` (5 min, was 1 min in v3)
7. `LoadingFallback` component mounts → `setInterval(DOM scan, 3000)`
8. `HealthMonitorStarter` mounts → `setTimeout(health check, 10000)`
9. `retryFailedOrders()` runs silently in background

2.2 Verdict: completes, but with 2 latent DOS vectors

Workflow completes successfully. The user sees products within ~300ms of first paint (cached), then sees the fresh catalog within ~1-2 seconds (Worker fetch). This is correct behavior. However, two latent issues will manifest at scale: (1) **R-2: bumpHitCounter is now called on every visitor** — at 50K visitors/day that's 50K KV writes/day, but the free-tier quota is 1K writes/day. The Worker will start returning 5xx errors after the quota is exhausted, breaking all catalog reads. (2) `LoadingFallback` still does 3-sec DOM scans (P-04 unfixed), which becomes a perf issue only at scale. At current 61-product scale, both issues are invisible. **They become critical at ~3K visitors/day (KV quota exhaustion in ~7 hours) or at 9,500 products (DOM scan perf hit).**

2.3 Specific v4 changes that affect this workflow

File	Change	Impact
use-catalog.ts:39	POLL_MS 60_000 → 300_000 (5 min)	Worker requests drop ~83% — big quota savings, OK for v3 fix P-04
use-catalog.ts:40	HIDDEN_POLL_MS 600_000 → 1_800_000 (30 min)	Background tabs poll 3× less frequently — good
worker/data-sync.js:201, 228,261,273,296,307	Cache-Control: no-store → public, max-age=300, stale-while-revalidate=600	Worker responses now cached at browser+CDN for 5 min. Dramatic quota savings, BUT stale data window up to 10 min.
worker/data-sync.js:515-527	bumpHitCounter sampling REMOVED — writes on every visitor	⚠ REGRESSION R-2: 50K writes/day = 50× over 1K/day quota. Catalog breaks within ~7h of typical traffic.
worker-client.ts (whole file)	Removed: refreshAfterAdminOp, triggerWorkerRefreshOnly, fetchCatalog(forceRefresh) cache logic simplified	Simpler code but lost the deduplication of admin-triggered refreshes. Multiple admin ops in rapid succession now each trigger separate syncs.
layout.tsx:82-83	Removed preconnect to Worker URL	Marginal perf regression — 1 extra DNS lookup on first Worker request

3. Workflow 2 — Admin Login (Auth Flow)

3.1 Trace

```
1. Admin navigates to https://soudmeco.pages.dev/#admin
2. AdminPanel component mounts (statically imported in page.tsx:16 – P-02 unfixed)
3. PasswordGate renders:
3a. Reads ADMIN_PASSWORD = BRAND.adminPassword (='dimou2411@dz' – S-01 unfixed)
3b. Renders password input + 'لوخد' button
4. Admin types password, clicks Submit:
4a. Compares value === ADMIN_PASSWORD (client-side string compare – S-01)
4b. If match: sessionStorage.setItem('soudmeco_admin_authed', '1')
4c. If no match: setError(true) → shows error message
5. On next render: useEffect checks sessionStorage for 'soudmeco_admin_authed'
5a. If === '1' → setAuthed(true) → renders admin panel
5b. Otherwise → PasswordGate remains

BYPASS PATHS (any one is sufficient):
• DevTools Console: sessionStorage.setItem('soudmeco_admin_authed', '1')
• DevTools Sources: search bundle for 'dimou2411@dz' → reveals password
• DevTools Sources: search bundle for 'adminPassword' → reveals BRAND.adminPassword
• curl: download the JS bundle, grep for the password string
```

3.2 Verdict: completes but provides zero real authentication

The auth 'flow' completes (admin enters password, sees admin panel), but it provides no actual security. The password is hardcoded in brand-config.ts:17 and shipped in every visitor's JS bundle. The auth check happens entirely client-side. An attacker doesn't even need the password — they can directly set the sessionStorage key. **This is identical to v3.** The team made zero changes to auth in v4. All 8 security findings from Phase 1 of my v3 audit (S-01 through S-08) remain unfixed.

3.3 What 'fixing this' would have looked like (constraint-aware)

My Smarter-Solutions addendum proposed: add POST /?action=auth to the Worker, which validates against env.ADMIN_PASSWORD (set via wrangler secret, never in the bundle). Worker returns the ADMIN_SECRET as a token. Admin panel stores the token in sessionStorage. All subsequent admin writes include X-Admin-Secret. This requires zero new infra and is ~30 lines of code change. v4 did not implement it.

4. Workflow 3 — Admin Uploads Product Image

4.1 Trace

1. Admin clicks Edit on a product → EditForm opens (admin-panel.tsx:207)
2. Admin drags/selects image file → handleFiles() (admin-panel.tsx:242)
3. `resizeImage(file, 850, 200_000)` – canvas resize, returns data: URL
4. `clientUploadImages([dataUrls], tempId)` (client-sheet.ts:417):
 - 4a. For each data: URL → `clientUploadImage()`
 - 4b. POST to `https://api.cloudinary.com/v1_1/anhvhy4j/image/upload`
 - `upload_preset: 'sourmdeco'` (unsigned – S-17 unfixed)
 - `public_id: filename`
 - `file: data URL`
 - 4c. Limited parallelism: 2 uploads at a time
 - 4d. On 400: retry once WITHOUT `public_id` (preset may not allow it)
 - 4e. Returns: `['https://res.cloudinary.com/anhvhy4j/image/upload/v123/...']`
5. `syncPhotos([...uploadedUrls])` – updates `draft.images`
6. Admin clicks Save → `handleSave(p)` (admin-panel.tsx:1219):
 - 6a. (NEW IN v4) `reverseOptimizeUrl(p.image)` – converts any local path back to Cloudinary URL before passing to `onUpsert`
 - 6b. (NEW IN v4) `p.images.map(reverseOptimizeUrl)` – same for images array
 - 6c. Calls `onUpsert(cleanProduct)` → `useCatalog.upsertProduct()`

WORKFLOW VERDICT: ☐ completes correctly. Image lands on Cloudinary. URL is preserved through to the sheet. The new `reverseOptimizeUrl` step ensures that even if the product had local paths (from a prior leak), they get converted back to Cloudinary URLs before the sheet POST.

4.2 Why this workflow is now correct (Cloudinary bug fixed)

In v3, the leak happened because `optimizeCloudinaryUrls` (called in `refresh()`) rewrote Cloudinary URLs to local paths on the React state object. When the admin later saved the product, that rewritten local path was sent to the sheet. **In v4, the admin panel explicitly reverses the rewrite before saving** (admin-panel.tsx:1228-1235). And then `use-catalog.ts:300-305` applies the reverse AGAIN before posting to Apps Script (defense-in-depth). This means the sheet ALWAYS receives a Cloudinary URL, never a local path. The leak is genuinely closed.

4.3 Remaining gaps in this workflow

ID	Severity	Issue	Location	Detail
G-01	HIGH	Cloudinary preset still unsigned	client-sheet.ts:285-407 + .env.production:3 (deleted but pattern remains)	Anyone can POST to api.cloudinary.com/v1_1/anhvhy4j/image/upload with upload_preset=sounddeco. Free image hosting abuse, billing quota exhaustion, illicit content under merchant's account.
G-02	HIGH	admin-panel.tsx statically imported by every visitor	src/app/page.tsx:16	All 1,596 lines of admin code (including resizeImage, PasswordGate, BRAND.adminPassword literal) ship to every visitor. ~30-50KB wasted + security exposure.
G-03	MEDIUM	MAX_FILE_SIZE check can crash the browser	admin-panel.tsx:60-64	15MB limit is enforced AFTER the file is selected. The browser may already have read the file into memory by then. Should be checked on the file input's onChange before any processing.
G-04	MEDIUM	Two reverseOptimizeUrl implementations (DRY violation)	use-catalog.ts:47 + admin-panel.tsx:1217	Both define the same function with the same logic. If one is updated and the other isn't, the leak comes back. Should be a single shared export in lib/products.ts.
G-05	LOW	Hardcoded Cloudinary cloud name in reverseOptimizeUrl	use-catalog.ts:51 + admin-panel.tsx:1221	Both hardcode 'anhvhy4j'. If the merchant ever rotates Cloudinary accounts, both need updating. Should read from process.env.NEXT_PUBLIC_CLOUDINARY_CLOUD_NAME.
G-06	LOW	No image-content validation (only file type)	admin-panel.tsx:67-73	Only checks file.type starts with 'image/'. A malicious SVG-with-embedded-script could pass (SVG is rejected, but other formats like image/webp with EXIF payloads are not stripped).

5. Workflow 4 — Admin Saves Product (with images)

5.1 Trace (the most critical workflow — was the source of the Cloudinary bug)

1. Admin clicks Save in EditForm → `handleSave(p)` (`admin-panel.tsx:1219`)
2. (NEW IN v4) Build `cleanProduct`:
`cleanProduct.image = reverseOptimizeUrl(p.image)` ← converts `/images/products/X` back to `https://res.cloudinary.com/.../X`
`cleanProduct.images = p.images.map(reverseOptimizeUrl)`
3. `await onUpsert(cleanProduct) → useCatalog.upsertProduct(product)` (`use-catalog.ts:279`)
4. Optimistic update:
 - 4a. `findIndex` of `product.id` in `productsRef.current`
 - 4b. If found: replace. If not: prepend.
 - 4c. Sort by `sortOrder`
 - 4d. `setProducts(optimisticNext) + saveCatalog(optimisticNext)`
5. `setSyncing(true)`
6. (NEW IN v4) Reverse-optimize AGAIN inside `upsertProduct`:
`cleanImages = product.images.map(reverseOptimizeUrl)` ← defense in depth
`cleanImage = reverseOptimizeUrl(product.image)`
7. `clientUploadImages(cleanImages, product.id)` — no-op for non-data: URLs
8. Build `SheetProduct` payload (`use-catalog.ts:316`):
`image: coverImage (= cleanImage, since uploads are no-op for URLs)`
`images: joinImageStrings(uploadedUrls)` — joined with `~~~`
9. `await clientUpsertProduct(sheetProduct)` (`client-sheet.ts:177`):
 - 9a. POST to `https://script.google.com/.../exec?action=product_create`
 - 9b. Body: `JSON.stringify(product)`, `Content-Type: text/plain; charset=utf-8`
 - 9c. (REGRESSION R-3) Returns `res.ok` ONLY — no longer accepts 302/405 as success
- △ If Apps Script returns 302→405 (its documented behavior), this returns FALSE
 → `upsertProduct` throws → admin sees 'Apps Script POST failed' error
 → BUT the product WAS actually saved before the redirect
 → admin retries → duplicate product created
10. On success:
`setTimeout(() => refresh().catch(() => {}), 100)` — non-blocking
 △ This re-fetches the WHOLE catalog (~2-5s) just to sync the change
 △ Was: `applyFreshCatalogAfterOp` (deduped, triggered Worker /refresh)
 △ Now: just calls `refresh()` — burns Worker quota unnecessarily
11. On failure:
 Rollback: `setProducts(previousProducts) + saveCatalog(previousProducts)`
 Throw error → admin sees toast

WORKFLOW VERDICT: △ MIXED. The Cloudinary leak is fixed (steps 2 + 6). But R-3 (step 9c) may cause false-failure on Apps Script POST, and the deletion of `applyFreshCatalogAfterOp` (step 10) means admin saves now trigger full catalog re-fetches instead of surgical Worker refresh.

5.2 The R-3 regression — does it actually break admin saves?

Apps Script's documented behavior when receiving POST + redirect:follow is to return 302 (redirect to script.googleusercontent.com) → 405 (Method Not Allowed on the redirect target). The v3 code at `client-sheet.ts:199` explicitly accepted 200, 302, and 405 as success — with a clear comment explaining that the POST actually succeeds before the redirect. **The v4 code at `client-sheet.ts:199` only accepts 200.** If Apps Script's redirect behavior is unchanged (which is highly likely — Google doesn't change this), then admin saves will now appear to fail even though they actually succeeded. The admin will see an error toast, retry, and create a duplicate product. **This is a behavior regression that I cannot verify without testing against the live Apps Script deployment, but the v3 code's comment strongly suggests it will manifest.**

Action needed: verify R-3 by testing admin save

Have the admin save any product on the v4 deployment. If they see 'Apps Script POST failed — product not saved to sheet' but the product IS in the sheet, R-3 is confirmed. Fix: restore the 302/405 acceptance in `client-sheet.ts:199` (one line: `return res.ok || res.status === 302 || res.status === 405;`).

5.3 The deletion of `applyFreshCatalogAfterOp` — quota impact

In v3, after every admin op, `applyFreshCatalogAfterOp()` ran a deduplicated Worker /refresh (1 KV read + 4 KV writes per op, deduplicated across rapid ops). The admin's optimistic update stayed in place. **In v4, this entire function is gone.** Instead, `upsertProduct` now just calls `refresh()` after 100ms, which re-fetches the WHOLE catalog from the Worker (1 KV read). This is actually LIGHTER on KV writes (good!) but heavier on Worker CPU (1 full catalog fetch per admin op vs. 1 tiny refresh call). At current 61-product scale, this is fine. At 9,500 products, it would be a problem.

6. Workflow 5 — Visitor Submits an Order (PII path)

6.1 Trace

```

1. Visitor adds products to cart (use-cart.ts – unchanged)
2. Visitor opens checkout modal → CodOrderForm (cod-order-form.tsx – 1,097 lines)
3. Visitor fills: fullName, phone (Algerian 0X XX XX XX XX), wilaya, commune, address
4. Visitor clicks 'بطل اديكأت' (Confirm Order):
4a. Form validates phone regex /^0[567]\d{8}$/
4b. POST to /api/order (NEXT.js Pages API edge route)
5. /api/order/route.ts receives POST:
5a. Validates phone + required fields (fullName, wilaya, delivery)
5b. Calls sheetSubmitOrder(payload) (sheet.ts:154)
6. sheetSubmitOrder:
6a. Builds URL: https://script.google.com/.../exec?action=order
6b. URLSearchParams with: product, quantity, price, shippingPrice, grandTotal,
    shippingCompanyLabel, fullName, phone, wilaya, commune, deliveryLabel, notes
6c. Truncates each field (fullName: 100, phone: 20, notes: 200, etc.)
6d. GET to Apps Script URL (NOT POST – S-08 unfixed)
  △ PII in URL params: ends up in browser history, server logs, proxy logs
  △ Violates Algerian Law 18-07 (S-PII-01 unfixed)
7. On success: NextResponse.json({ ok: true })
8. On failure:
8a. console.error('[api/order] FAILED:', {fullName, phone, wilaya, grandTotal})
  △ PII logged to Cloudflare edge logs (PII-02 unfixed)
8b. Returns NextResponse.json({ ok: true, warning: 'order_queued' })
  △ Customer sees success even though order didn't reach sheet (S-18 unfixed)
9. Browser receives response:
9a. If warning: 'order_queued' → clientSubmitOrder retries via client-sheet.ts
9b. If retry fails → addFailedOrder() stores in localStorage with full PII
  △ Plaintext, never expires (PII-03 unfixed)
10. retryFailedOrders() runs on next page visit – retries silently

```

WORKFLOW VERDICT: ☐ completes but with PII violations at every step.

The order reaches the sheet (eventually), but PII leaks through:

- URL params (browser history, server logs, proxy logs)
- console.error (Cloudflare edge logs)
- localStorage (plaintext, indefinite)
- Apps Script execution logs

Four PII copies, three of which leak. ALL findings from Phase 3 of the v3 audit remain unfixed.

6.2 Why this workflow is the most dangerous one in v4

Order submission is the one workflow that touches real customer PII (name + phone + address). Every other workflow is operational (catalog, images, admin state). v4 made zero changes to this workflow. Every PII finding from the v3 audit (PII-01 through PII-08) stands exactly as written. Under Algerian Law 18-07, this is a reportable breach waiting to happen. **If the site has been live and processing orders, the merchant is currently in violation of data-protection law.** The Cloudinary bug was an operational inconvenience; this is a legal exposure.

6.3 What the constraint-aware fix looks like (from the Smarter-Solutions addendum)

Move order submission to the Worker. Browser POSTs PII as JSON body to `/?action=submit_order` (with X-Admin-Secret NOT needed — orders are public). Worker validates input, forwards to Apps Script as POST body (not URL params), strips secret. Apps Script `doPost` writes to sheet. PII never appears in a URL. Failed orders encrypted with Web Crypto API (AES-GCM) using session salt from `sessionStorage`. Nightly Apps Script trigger deletes orders older than 90 days. Total cost: \$0, zero new accounts. ~50 lines of code change.

7. Workflow 6 — Admin Deletes a Product

7.1 Trace

```

1. Admin clicks trash icon in admin-panel.tsx product row
2. deleteProduct(id) (use-catalog.ts:370):
2a. Find product in productsRef.current → save for rollback
2b. Optimistic: filter out product → setProducts(next) + saveCatalog(next)
2c. setSyncing(true)
2d. clientDeleteProduct(id) (client-sheet.ts:209):
  - GET to https://script.google.com/.../exec?action=product_delete&id=...
  △ GET-based mutation (S-06 unfixed) – CSRF-friendly, prefetch-friendly
2e. On success: setTimeout(() => refresh().catch(() => {}), 100)
  △ (same as upsertProduct – full catalog refetch, no dedup)
2f. On failure: rollback + throw
3. Worker /refresh is NOT triggered (applyFreshCatalogAfterOp was deleted in v4)
  △ Other visitors will see the deleted product for up to 5 min (their next poll)
  △ Or up to 10 min if their poll hit the new Cache-Control max-age=300 window

WORKFLOW VERDICT: □ completes correctly for the admin (optimistic UI works).
  △ Other visitors see stale data for 5-10 min (was 1-5 min in v3 due to
  applyFreshCatalogAfterOp triggering an immediate Worker /refresh). This is
  a UX regression for multi-admin scenarios (admin A deletes, admin B's
  browser still shows it).
```

7.2 The 'multi-admin stale data' problem

In v3, after admin A deleted a product, the Worker /refresh was triggered immediately, so admin B (if polling) would see the deletion within ~5 seconds. In v4, that trigger is gone. Admin B's browser will see the deleted product for up to 5 minutes (their next 5-min poll) or up to 10 minutes (if their last poll hit the 5-minute Cache-Control window). For a single-admin store this is fine. For a multi-admin store or for the merchant checking 'did I really delete that?' from another device, it's confusing.

8. Workflow 7 — Worker Cron Sync (every 1 minute)

8.1 Trace

```

1. Cloudflare cron fires every 1 minute (worker/wrangler.toml crons = ['* * * * *'])
  △ Code comments still say 'every 5 minutes' – schedule mismatch unfixed (Q-06)
2. Worker scheduled() handler fires:
  2a. ctx.waitUntil(syncData(env)) – non-blocking
  2b. try/catch – never throws (so cron always succeeds)
3. syncData(env) (worker/data-sync.js:361):
  3a. Read env.APPS_SCRIPT_URL – if missing, return error
  3b. Read existing __hashes + __meta from KV (2 reads)
  3c. fetchFromAppScript(URL, 'products') – 3 retries, 10s timeout each
  3d. fetchFromAppScript(URL, 'stock') – same
  3e. Both in parallel (Promise.allSettled)
  3f. Hash the responses (djb2)
  3g. Compare to old hashes – detect changes
  3h. KV writes (Promise.allSettled):
      - 'products' key (with 1h TTL)
      - 'stock' key (with 1h TTL)
      - '__hashes' key (no TTL)
      - '__meta' key (lastSync, lastSyncAttempt, lastChange, kvHits, failures)
  = 4 KV writes per cron cycle
4. Cron math:
  - 1,440 cron cycles/day × 4 writes/cycle = 5,760 KV writes/day
  - Free-tier limit: 1,000 writes/day
  △ 5.76× OVER the quota, EVEN WITHOUT any visitor traffic
  △ PLUS R-2 (every visitor writes 1 KV) = additional 50K writes/day at scale
  △ TOTAL: ~55,760 writes/day vs 1,000 limit = 55.76× OVER

```

WORKFLOW VERDICT: △ cron itself completes correctly, but the quota math is CATASTROPHIC. The free-tier KV write quota will be exhausted within ~25 minutes of typical traffic. After that, all Worker writes fail silently, the catalog freezes at whatever was last successfully written, and visitors see stale data until the next billing cycle resets the quota.

8.2 Why v3 didn't have this problem

v3 had three protections that v4 deleted: (1) **Hit-counter sampling** (worker/data-sync.js:515-527) — only 1-in-50 visitors triggered a KV write, reducing visitor-driven writes from 50K/day to ~1K/day. (2) **/refresh rate limiting** (worker/data-sync.js:130-160) — max 1 refresh per second, preventing admin-triggered write bursts. (3) The cron was always going to exceed quota at 1-min interval, but the sampling kept visitor-driven writes low enough that the cron's writes were the dominant cost — and Cloudflare appears to soft-cap (continue serving reads, fail writes silently) rather than hard-block. **In v4, all three protections are gone.** The cron alone is 5.76× over quota. The visitor traffic adds another 50×. The Worker will hit the wall very quickly.

8.3 The fix (5 minutes, \$0)

```
# Restore the sampling in worker/data-sync.js (replace lines 485-499):
async function bumpHitCounter(env) {
  try {
    // Sample: only bump ~2% of requests (1 in 50)
    if (Math.random() > 0.02) return; // skip 98% of the time
    const meta = await env.CATALOG_KV.get("__meta", "json").catch(() => null);
    const next = {
      lastSync: meta?.lastSync || Date.now(),
      lastSyncAttempt: meta?.lastSyncAttempt || Date.now(),
      lastChange: meta?.lastChange || Date.now(),
      kvHits: (meta?.kvHits || 0) + 50, // scale up by 50x to approximate
      consecutiveFailures: meta?.consecutiveFailures || 0,
    };
    await env.CATALOG_KV.put("__meta", JSON.stringify(next));
  } catch {}
}

# AND reduce cron to every 5 min (was 1 min):
# worker/wrangler.toml
[triggers]
crons = ["*/5 * * * *"] # was "* * * * *"

# Result: cron writes = 288/day × 4 = 1,152 → still slightly over, but
# Cloudflare soft-caps. Visitor writes = 50K × 0.02 = 1,000. Total
# ~2,152 writes/day = 2.15× quota. Better but still over.
#
# Better: skip cron writes when data unchanged (hashes match). v3 already
# does this at line 413 – but it still writes __meta. Fix: only write
# __meta every 10th cycle, or move meta to Durable Objects (paid).
```

9. Workflow 8 — Background Polling + Visibility Change

9.1 Trace

```
Initial mount (use-catalog.ts:173):
1. loadCatalog() synchronous (cached data)
2. setProducts(sorted) + setHydrated(true) + setLoading(false)
3. setTimeout(refresh, 300) – delayed first fetch
4. scheduleNext() – setInterval(refresh, 300_000) (5 min, was 1 min in v3)
5. loadCatalogAsync() – IndexedDB fallback check
6. loadImageManifest() – then re-apply optimizeCloudinaryUrls
7. retryFailedOrders() – silent
8. document.addEventListener('visibilitychange', onVisibility)

On visibilitychange (use-catalog.ts:252):
- If tab visible (was hidden): refresh() fires immediately
- scheduleNext() – re-evaluates interval (5 min visible, 30 min hidden)

Per poll (every 5 min):
1. Optimistic setProducts(cached) – instant paint
2. fetchProducts() → Worker /?action=catalog
  - Worker reads KV (1 read)
  - (REGRESSION R-2) bumpHitCounter writes KV on every poll
  - Returns stringified products + stock CSV
3. Apply optimizeCloudinaryUrls → saveCatalog → setProducts

Per poll math (50K visitors, 4-min avg session):
v3: 1 poll/min × 4 min × 50K = 200K Worker requests/day = 2× over quota
v4: 1 poll/5min × 4 min × 50K = 40K Worker requests/day = 40% of quota □
v4 KV reads: 40K (40% of quota) □
v4 KV writes: 40K (REGRESSION – was ~1K in v3 due to sampling) = 40× over □

WORKFLOW VERDICT: □ polling itself is well-tuned in v4 (5-min interval is
the right call). □ But every poll now triggers a KV write (R-2), which
alone is 40× over the daily write quota.
```

9.2 The use-stock.ts duplicate-poll problem (still unfixed)

use-stock.ts (434 lines, unchanged in v4) has its OWN setInterval at 60s (P-09 unfixed). It calls fetchStockCsv() which internally calls fetchCatalog() — same data, fetched twice. v4 deleted some code from worker-client.ts but kept fetchStockCsv. The two hooks still poll independently. At 50K visitors × 2 polls/visit = 100K requests/day — just from stock polling alone, that's the entire Worker request quota.

10. Workflows 9-16 — Compacted Verdicts

#	Workflow	Verdict	Notes
9	Admin reorders products (up/down arrows)	☑ Completes	moveProduct now applies reverseOptimizeUrl (use-catalog.ts:479-481) — no leak. Both swapped products are POSTed to Apps Script in parallel. ☑
10	Admin clicks 'Reset catalog' button	⚠ Completes but unsafe	resetCatalog calls clientResetProducts which GETs ?action=product_reset — wipes entire catalog via GET (S-06). No confirmation token. No auth. Anyone can trigger via URL.
11	Visitor browses products in cart	☑ Completes	use-cart.ts unchanged. localStorage-based, works offline. No issues.
12	Image render on product card	☑ Completes	product-image.tsx unchanged. The onError → buildCloudinaryFallback path is still present but rarely triggered now that the leak is fixed. P-05 (no srcset) still unfixed.
13	Failed-order retry queue	⚠ Completes but PII leaks	retryFailedOrders runs on every page mount. PII stored in localStorage plaintext, indefinite retention (PII-03). No encryption (P3.6 unfixed).
14	Worker /refresh (admin manual trigger)	⚠ Completes but DOSable	(REGRESSION R-1) Rate limiter deleted. Anyone with the public admin secret can spam /refresh. Each call = 4 KV writes. 250 calls in 5 min = quota exhausted.
15	Worker /health endpoint	⚠ Completes but leaks metadata	Unauthenticated. Returns lastSync, productCount, kvHits, consecutiveFailures. Reconnaissance aid for timing attacks. (S-14 unfixed.)
16	Auto-sync (GitHub Actions, daily)	☑ Completes	scripts/auto-sync.py unchanged. Now uses correct SCRIPT_DIR pattern (P2.3 fix). Manifest grew from 23 to 87 files — sync is working. ☑

11. The 4 New Regressions v4 Introduced

11.1 R-1: Worker /refresh rate limiter removed (CRITICAL)

What was removed: worker/data-sync.js lines 126-160 (the 1-second cooldown block). In v3, every /refresh call checked `__meta.lastSyncAttempt` and rejected calls within 1 second of the last attempt. This prevented quota abuse (each /refresh = 4 KV writes, max 1K writes/day quota).

Why it matters: The admin secret is inlined into the client bundle (NEXT_PUBLIC_WORKER_ADMIN_SECRET). Anyone can extract it and spam /refresh. Without the rate limiter, an attacker can call /refresh 1,000 times in 4 minutes and exhaust the entire day's KV write quota. After that, the cron can't write fresh data, the catalog freezes, and visitors see stale products until the next billing cycle.

Fix: Restore the deleted block (30 lines). Or, smarter: use a Cloudflare Worker in-memory rate limit (per-isolate counter) to avoid the KV read entirely. Even smarter: move /refresh to require a one-time CSRF token issued at admin login (changes the threat model from 'anyone with the secret' to 'anyone who logged in as admin').

11.2 R-2: KV hit-counter sampling removed (CRITICAL)

What was removed: worker/data-sync.js lines 515-527 (the `if (Math.random() > 0.02) return; sampling`). In v3, only 1 in 50 visitors triggered a KV write to bump the hit counter. The displayed count was scaled up by 50× to approximate the real number. This kept visitor-driven KV writes at ~1K/day (right at the quota limit).

Why it matters: v4's bumpHitCounter writes on EVERY visitor. At 50K visitors/day = 50K writes/day. Free-tier quota = 1K/day. **The Worker will start failing all writes within ~30 minutes of typical traffic.** Reads continue to work (separate quota), so visitors still see the catalog — but the cron can't update it, and admin /refresh calls fail silently. The catalog freezes at whatever was last successfully written.

Fix: Restore the sampling. Even better: delete bumpHitCounter entirely — it's a vanity metric with no operational value. The team can get real analytics from Cloudflare's dashboard (Workers → Metrics tab) for free, with zero KV writes.

11.3 R-3: clientUpsertProduct no longer accepts 302/405 (HIGH)

What changed: client-sheet.ts:199 (now ~line 193 in v4). v3 explicitly accepted 200, 302, AND 405 as success: `return res.ok || res.status === 302 || res.status === 405`; — with a 5-line comment explaining why (Apps Script POSTs return 302→405 after redirect, but the product IS saved before the redirect). v4 simplified to just `return res.ok`;

Why it matters: If Apps Script still exhibits the 302→405 behavior (which is highly likely — Google doesn't change this), admin saves will now appear to fail. The admin sees an error toast, retries, and creates a duplicate product. I cannot verify this without testing against the live Apps Script deployment, but the v3 code's explicit comment + 5-line justification strongly suggests the behavior is real.

Fix: Restore the 302/405 acceptance. One line: `return res.ok || res.status === 302 || res.status === 405`;

11.4 R-4: products.json.backup-* file leaked into public/data (MEDIUM)

What's there: public/data/products.json.backup-1787141289 (67 KB, 80 products) is in the v4 zip. This file is in /public/data/, which means it's downloadable by anyone at <https://soumdeco.pages.dev/data/products.json.backup-1787141289>. It appears to be a backup taken before the repair-images.py script ran — it contains 50 products with local paths and 30 with Cloudinary URLs.

Why it matters: (1) It exposes the pre-repair state of the catalog publicly. (2) It's 67 KB of dead weight on every deploy. (3) The Cloudflare Pages _headers file at /data/products.json sets max-age=60, stale-while-revalidate=300 — but the backup file has no cache rule, so it's served with default caching. (4) Future builds will keep shipping this file unless it's deleted or .gitignore'd.

Fix: Delete the file. Add *.backup-* to .gitignore. Also add a CI step that fails the build if any *.backup-* file is found in /public.

12. v4-Specific Remediation Roadmap

This roadmap assumes v4 is the new baseline. It incorporates everything from the v3 audit + Smarter-Solutions addendum + the 4 new v4 regressions. Order is by urgency.

Phase 0 — Block v4 self-DOS (TODAY, 30 minutes, \$0)

0.1 RESTORE the /refresh rate limiter in worker/data-sync.js (30 lines, was at lines 126-160 in v3).

0.2 RESTORE the bumpHitCounter sampling (1 line: `if (Math.random() > 0.02) return;`). OR delete bumpHitCounter entirely (vanity metric, get real analytics from Cloudflare dashboard).

0.3 RESTORE the 302/405 acceptance in client-sheet.ts:193 (1 line: `return res.ok || res.status === 302 || res.status === 405;`).

0.4 DELETE public/data/products.json.backup-1787141289. Add *.backup-* to a new .gitignore file.

0.5 Reduce cron from 1-min to 5-min in worker/wrangler.toml (1 char: `*/5 * * * *` instead of `* * * * *`). Saves 80% of cron-driven KV writes.

Phase 1 — Stop the bleeding (THIS WEEK, 1 day, \$0)

1.1 Rotate ALL secrets (Apps Script URL, Cloudinary preset, Worker ADMIN_SECRET, admin password). Assume all current secrets are burned.

1.2 Add .gitignore with .env*, *.backup-*, node_modules/, .next/, .wrangler/

1.3 Remove adminPassword from brand-config.ts:17. Add POST /?action=auth to Worker. Admin panel fetches token, stores in sessionStorage.

1.4 Add X-Admin-Secret header check to all Worker mutating endpoints (write_product, delete_product, reset_products, upload_image_proxy).

1.5 Move admin writes from browser→Apps Script to browser→Worker→Apps Script. Delete the hardcoded SHEET_BASE_URL in sheet.ts:8-9.

1.6 Convert Apps Script ?action=product_reset and ?action=product_delete from GET to POST.

Phase 2 — PII compliance (THIS WEEK, 1 day, \$0)

2.1 Move order submission to Worker /?action=submit_order. Browser POSTs PII as JSON body. Worker forwards to Apps Script as POST body (not URL params). PII never in URLs.

2.2 Stop logging PII in /api/order/route.ts:58-64. Replace `console.error({fullName, phone, ...})` with `console.error({status, orderId})`.

2.3 Encrypt failed-orders queue in localStorage using Web Crypto API (AES-GCM with session salt from sessionStorage). ~15 lines of code.

2.4 Add nightly Apps Script trigger to delete orders older than 90 days. Achieves Algerian Law 18-07 retention compliance.

2.5 Add honeypot + time-to-submit check + probabilistic KV-counter rate limit to order form.

Phase 3 — Performance & quality (NEXT 2 WEEKS, \$0)

- 3.1 Dynamic import AdminPanel (ssr: false). Saves ~30-50KB gzipped + removes adminPassword from non-admin bundles.
- 3.2 Move SEED_PRODUCTS to /public/seed-products.json. Lazy fetch only when other sources fail.
- 3.3 Delete 47 unused shadcn UI components + matching deps from package.json.
- 3.4 Convert Hero, Categories, BrandStory, SiteFooter to server components.
- 3.5 Merge use-stock into use-catalog. Single interval, single fetch.
- 3.6 Delete LoadingFallback.tsx entirely. Skeleton + ErrorBoundary already cover it.
- 3.7 Memoize ProductCard + ProductImage with React.memo.
- 3.8 Consolidate the 2 reverseOptimizeUrl functions into 1 shared export in lib/products.ts.
- 3.9 Remove typescript.ignoreBuildErrors: true. Fix every TS error.
- 3.10 Restore ESLint rules incrementally. Start with no-explicit-any: warn, exhaustive-deps: warn.

Phase 4 — Scale preparation (NEXT MONTH, \$0)

- 4.1 If approaching 9,500 products: shard the KV (96 shards × ~200KB each). Worker CPU drops from 200ms to 2ms per request.
- 4.2 If approaching 9,500 products: create second free Cloudflare Pages project for image overflow.
- 4.3 Add @tanstack/react-virtual to AllProducts rows + admin panel list.
- 4.4 Move catalog parse/sort to Web Worker. Saves 200-600ms main-thread block per poll.
- 4.5 Add Vitest unit tests for: Worker data-sync.js, use-catalog.ts, product-image.tsx.
- 4.6 Make repo PUBLIC (it's a generic Next.js storefront). Unlimited GitHub Actions minutes.
- 4.7 Add 3-5 Playwright e2e tests: admin login + product create + product delete + order submission.

13. Final Verdict on v4

TL;DR

v4 is a focused fix-release that successfully closes the Cloudinary image-corruption bug (the headline issue from the v3 audit). The team added `reverseOptimizeUrl` in two places, wrote and ran a repair script, and fixed the `build-image-manifest.py` paths. That's real, verifiable progress. But v4 also introduced 4 new regressions, two of which (R-1 and R-2) will cause the Worker to DOS itself within 30 minutes of typical traffic by burning the KV write quota 50× over. The admin password is still in the client bundle. The admin secret is still inlined via `NEXT_PUBLIC_`. PII still goes in URL params.

Recommendation: do not deploy v4 to production without completing Phase 0 (30 minutes of work to restore the deleted safeguards). After Phase 0, v4 is materially better than v3.

What's genuinely improved in v4

The Cloudinary bug is fixed. I verified the data: `products.json` now has 0 local paths (was 44 of 60 in v3). The `reverseOptimizeUrl` function is applied at two layers (`admin-panel.tsx` + `use-catalog.ts`) — defense in depth. The `build-image-manifest.py` is now CI-safe (`SCRIPT_DIR` pattern) and fails loud on empty manifest. The polling interval was correctly increased from 60s to 300s. The Worker now sends `Cache-Control: max-age=300` headers, which dramatically reduces Worker CPU and request quota usage (when combined with the 5-min poll). These are real, correct fixes.

What's regressed in v4

The Worker now writes KV on every visitor (R-2, sampling removed). **The Worker /refresh endpoint has no rate limit** (R-1, limiter removed). **Admin saves may report false failures** (R-3, 302/405 no longer accepted). **A backup products file leaked into /public/data/** (R-4). Of these, R-1 and R-2 combined are the most dangerous — they will burn the entire KV write quota within 30 minutes of typical traffic, freezing the catalog for all visitors until the next billing cycle.

What's still wrong (unchanged from v3)

Zero real authentication anywhere. Admin password still in bundle. Admin secret still inlined via `NEXT_PUBLIC_`. All mutating API routes still unauthenticated. PII still goes in URL params. PII still logged to server logs on failure. Failed orders still in `localStorage` plaintext forever. Cloudinary preset still unsigned. 47 unused shadcn UI components still in the repo. `LoadingFallback` still does 3-sec DOM scans. TypeScript errors still ignored at build. ESLint still has every meaningful rule off. No tests. No CI. **All 75+ open findings from the v3 audit + Smarter-Solutions addendum still apply.**

Recommendation

Do not deploy v4 to production as-is. Implement Phase 0 first (30 minutes — restore the deleted safeguards). After Phase 0, v4 is materially better than v3 and can deploy. Then proceed with Phase 1 (security) and Phase 2 (PII compliance) within the next week. The Cloudinary fix is real and should ship — but not at the cost of a self-DOSing Worker.

Workflow-integrity audit performed by Super Z (GLM) on soumdeco-magic-v4-final. Formidable-prompt method: every workflow traced end-to-end from origin to completion, no skipping. 16 workflows analyzed.

5 v3 findings confirmed fixed, 4 new regressions identified, 75+ findings still open. Companion to the Brutally Honest Audit Report (v3) and the Smarter Solutions addendum.